

Architect — compiled path

Compiled from `/home/dnikolic/Documents/Learning/Plan učenja/paths/Architect`. Canonical sources stay in place.

Phase — 01-system-design-foundations

Directory: `01-system-design-foundations/`

Source: `01-system-design-foundations/01-scope-architect-thinking-under-constraints.md`

Unit 1 — Scope: architects decide under constraints

Suggested cadence (informational): prefixes are topic order—not a timetable. Pace by depth, not by calendar myths.

What an architect optimises

An architect is not “the cleverest programmer.” They make **technical decisions under constraints**:

budget • schedule • team skills • latency • scale • ops • security • compliance • maintainability • risk

Thinking shift to internalise:

- From “make it run” toward **years** of viable ownership (“how it lives for ~3 years”).
- From “modernest stack” toward **least-bad compromise** (`tradeoff`).
- From “anything is possible” toward **articulated boundaries**.

Spine project (repeatable backbone)

Reuse a coherent reference topology when exercises mention it (Symfony + Go worker + Postgres + Redis + queue gateway). Substitute equivalents if needed, but preserve **explicit boundaries**.

Deliverables (discipline across this folder)

Produce **diagrams, constraints lists, failure hypotheses, numbers**—not slide-only rectangles.

Source: `01-system-design-foundations/02-from-problem-to-failure-models.md`

Unit 2 — Problem → requirements → constraints → trade-offs → failure model

Mental choreography (adapt names, keep structure):

Problem statement
Functional requirements
Non-functional requirements (NFRs)
Constraints (hard vs soft)
Design options (few, not infinitely many)
Comparable trade-offs (what you buy / what you pay)
Explicit failure scenarios + detection + rollback posture

Practice

Pick an existing codebase (Symfony monolith referenced in source—or your job system). Produce:

```
functional vs non-functional requirements table
explicit scale assumptions ("100 users", "100k concurrent sessions", ...)
explicit failure hypotheses (dependency X dies, hotspot key, operator misconfig...)
```

Lab (blog trajectory)

Contrast **three** evolutionary loads for the same product story:

```
~100 concurrent writers/readers caricature humble start
mid-scale traffic & storage amplification
million-user read-heavy skew with operational maturity expectations
```

State **what subsystem changes first** each time—not “everything becomes microservices magically.”

Source: `01-system-design-foundations/03-functional-vs-non-functional-requirements.md`

Unit 3 — Functional vs non-functional requirements (measurable envelopes)

Senior engineers ship features—architects keep **measurable NFR envelopes** aligned with risk.

Contrast deliberately:

```
Functional behaviours (capabilities users/companies visibly obtain)
Non-functional envelopes (latency, uptime, correctness under concurrency, auditability, sovereignty...)
Trade-off ledger (what improving one NFR costs another dimension)
```

Practice

For a representative HTTP API (Symfony monolith acceptable), produce three parallel lists:

1. Functional requirements (atomic, testable statements).
2. Non-functional requirements with **numbers** (even rough).
3. Trade-off notes where numbers conflict (cost vs latency vs complexity).

Interview drill

Name one NFR you would **not** promise on day one—and the **metric** that would force reopening the decision.

Source: `01-system-design-foundations/04-capacity-estimation-and-sensitivity.md`

Unit 4 — Capacity estimation: back-of-envelope numbers that prevent fantasy designs

Use explicit toy scenario from source:

```
100k users • 10 req/s • 1 MB payload • 30-day retention window (example only)
```

Derive defensible **orders-of-magnitude** for:

storage growth
egress / ingress bandwidth attention points
database row & index pressure intuition

Document assumptions table (peak factor, compression, dedupe, replication multipliers).

Lab deliverable

One-page calculation sheet + **sensitivity table** (“if payload 2× larger, what breaks first?”).

Source: [01-system-design-foundations/05-monolith-modular-monolith-and-microservices-tradeoffs.md](#)

Unit 5 — Monolith, modular monolith, microservices: when boundaries earn their complexity

Shape	Wins	Taxes
Monolith	fast iteration, simpler deploy early	coupling hotspots if boundaries ignored
Modular monolith (bounded contexts in one deployable)	domain clarity without network partitions	requires strict module rules & reviews
Microservices (only when triggers exist)	selective isolation of blast radius / scaling axes	contracts, observability, deploy complexity multiply

Practice

Annotate candidate seams in a Symfony-style monolith (`order`, `billing`, `user`, `notification`). For each candidate extraction to a separate service, list **what you buy** and **what you pay** (ops, data consistency, latency).

Interview drill

What **organisational or technical trigger** would make you accept distributed transactions / sagas instead of pretending one SQL transaction saves you?

Source: [01-system-design-foundations/06-async-gateway-failures-and-capstone-system-sketch.md](#)

Unit 6 — Sync vs async, gateway boundaries, failure thinking, integration lab

Sync vs async

Contrast **request/response** paths vs **queue-mediated** side effects (email, analytics). Address timeout ownership, user-visible vs background responsibilities.

API gateway & ownership

Gateway responsibilities (auth termination, routing, rate limits, WAF adjacency) vs domain services owning business rules—avoid “smart gateway / dumb services” vs “dumb gateway / duplicated auth mess” extremes without conscious policy.

Failure thinking (non-optional)

For each dependency (Redis, DB, DNS latency, third-party HTTP), document:

detection signal
degradation / fallback options
recovery window expectations
cost of being wrong

Capstone diagram (source Faza 1 integracioni lab)

Sketch end-to-end:

Frontend → Gateway → Symfony API → Go worker → Redis → Postgres

For each labelled concern, write **≥1 concrete sentence** (not a word on a slide):

- **Scale** — which component hits ceiling first under your assumptions.
 - **Failure** — one plausible outage and observable symptom.
 - **Monitoring** — the minimum signal proving health vs user pain.
 - **Backup / restore** — RPO/RTO statement (even qualitative).
 - **Security** — one trust-boundary assertion.
 - **Deployment** — one rollback / rollout constraint you accept.
-

Phase — 02-distributed-systems-architecture

Directory: `02-distributed-systems-architecture/`

Source: `02-distributed-systems-architecture/01-scope-failure-first-distributed-thinking.md`

Unit 1 — Scope: distributed-first failure imagination

Suggested cadence (informational): the source roadmap used ~twelve deepening blocks (~1 h/day).
Folder order ≠ calendar.

Outcome shift (**Faza 2** source): evolve from **"I finished an RPC"** toward **"I know what breaks when silence, slowness, or lies return"**.

Vocabulary anchors (articulate distinctions, don't meme patterns):

latency ceilings • cascading timeouts • retry policies & storms
idempotency & dedupe realism • consistency boundaries • partition tolerance humility (CAP caricature responsibly)
circuit breakers • bulkheads • saga/outbox choreography • duplicated events • ordering hotspots
backpressure (& operator visibility of backlog age) • blast radius narration

Practice spine topology (repeat deliberately)

Reuse exercise stack when relevant: **Gateway** → **Symfony API** → **Redis queue/worker** → **Postgres** → **eventual Go-worker**—adapt equivalent names consciously if your stack differs.

Cross-link voluntarily: parallels exist in `paths/Go/12-*`, `paths/Go/14-*` without duplicating full engineering depth repeatedly—this track stays **architecture reasoning**.

Source: `02-distributed-systems-architecture/02-core-practices-timeouts-retries-idempotency.md`

Unit 2 — Core practices: timeouts, retries, idempotency, inconsistency realism

Timeout budgets across hops

Design a **tiered budget** edge → gateway → service → downstream → DB—not a uniform deep stack of identical long timeouts multiplying queue buildup.

Explain what happens socially when downstream DB spikes latency and budgets are dishonest upstream.

Retries vs retry storms

List conditions when automated retries help vs harm (dependency already failing + amplified load). Tie to **bounded backoff** + **jitter** + **max attempts** motifs.

Explicitly anchor **financial** / **payment** ambiguity windows to idempotency keys (cross-link thinking to payment domains—no implementation mandate here).

Duplicated events & ordering hotspots

Enumerate how duplicate **OrderPaid**, **OrderCreated**, retries at-least-once semantics surface—and how consumer design must reconcile (dedupe ledger / idempotent semantics / partitioning keys bridging Kafka-thinking forward areas).

Unit 3 — Labs & integration capstone (distributed systems area)

Produce **three** short failure drills (diagram + bullets each):

1. **Queue/worker disappearance** — queue age metrics, UX degradation posture, remediation ownership.
2. **Database slowdown** — pool exhaustion story, cascading timeout propagation.
3. **Ambiguous payment RPC** — idempotency keys + reconciliation—not hope.

Capstone topology (reuse source stack)

Sketch & narrate:

```
Gateway → Symfony API → Redis queue/stream surface → Go worker → Postgres
```

Annotate **retry** / **timeout** / **monitoring** / **failure recovery** / **consistency realism** consciously—not slide-only labels without sentences.

Interview rehearsal

Five sceptical questions you'd ask **before** “we need microservices to fix outages.”

Phase — 03-storage-architecture

Directory: 03-storage-architecture/

Source: 03-storage-architecture/01-scope-why-we-store-this-way.md

Unit 1 — Scope: storage reasoning beyond “tables exist”

Suggested cadence (informational): source **Faza 3** pacing is guidance only.

Outcome shift: evolve from "where we store" toward "how storage semantics shape uptime, correctness, latency, ops, and economics".

Vocabulary anchors (apply concretely, not buzzwords):

- OLTP vs OLAP workloads (shape queries & tooling differently)
- Indexing & selective hotspot awareness (hot partitions / skewed keys)
- Replication & read replicas (staleness budgeting honestly)
- Consistency models articulated per operation—not one global slogan
- Sharding / partitioning motivations & cross-shard saga pain previews
- Retention, archival lifecycle, backup/restore (RPO/RTO conscious)
- Ownership of schemas & bounded contexts bridging modular monolith ideas
- Write amplification intuition (especially LSM-heavy stores when relevant)

Practice spine

Return to **Symfony + Postgres** reference—even if hypothetical—articulate **migration risk**, **index strategy**, **replication readiness** as architecture outcomes.

Source: 03-storage-architecture/02-labs-trade-offs-and-review-checklist.md

Unit 2 — Labs, trade-offs, readiness checklist

Design labs (minimum three documented)

Author short studies:

- Replication + read scaling** posture for read-heavy dashboards vs authoritative writes—state staleness UI expectations explicitly.
- Partitioning / sharding trigger** analysis: when single-node limits vs operational tax justifies split—include cross-shard query pain honesty.
- Backup/restore rehearsal narrative** (even tabletop): RPO/RTO targets + failure injection story.

Trade-off matrix (template)

Decision	Gain	Cost	Kill criteria (when you’d undo)
Add read replica			
Introduce caching layer			
Shard by tenant			

Interview rehearsal

Explain **one** storage choice you’d delay until metrics prove necessity—name the metric.

Phase — 04-search-architecture

Directory: 04-search-architecture/

Source: 04-search-architecture/01-scope-search-as-owning-system.md

Unit 1 — Scope: search as a subsystem, not SQL hacks

Suggested cadence (informational): ordering is thematic (04-* follows storage), not a timetable.

Treat **search** as an operational subsystem with its own failure modes—not “bigger Postgres **ILIKE**”.

Architectural distinctions

Full-text retrieval vs curated filters vs hybrid workloads (facet + text)
Ranking / relevance vs deterministic ordering (explain when each dominates)
Indexing lag & eventual freshness (explicit user-visible behaviour)
Fan-out costs & abusive query shaping (timeouts, quotas, sandboxing costly paths)
Operational ownership (index rebuild windows, versioning, synonym/dictionary churn)
Synchronisation honesty (dual-write pitfalls vs ingestion pipeline / CDC / outbox-style publication-pick consciously)

Practice spine

Relate **Symfony API + Postgres source-of-truth** to downstream **search index** with rollback story if indexer corrupts—not “silent divergence forever.”

Source: 04-search-architecture/02-labs-trade-offs-and-integration-checklists.md

Unit 2 — Labs & trade-offs: search ingestion, staleness, and failure containment

Lab A — ingestion contract

Specify how documents/records enter the search index:

event-triggered incremental updates vs periodic bulk rebuild trade spectrum
ordering / duplicate resilience (tie to distributed ingestion patterns)
explicit schema/version field for rollback & reindex choreography

Lab B — consistency narrative

Produce user-visible statement for **freshness SLA** (“search may lag writes by $\leq x$ minutes under normal churn”) tied to alerting if lag diverges dangerously.

Review checklist

Risk	Evidence you'd insist on
Hot queries overload cluster	latency metrics, shedding policy
Index skew / noisy neighbours	infra isolation story
PII leakage in index	field allow-lists
Divergent truth Postgres vs search	reconciliation job / breaker pattern

Phase — 05-caching-architecture

Directory: `05-caching-architecture/`

Source: `05-caching-architecture/01-scope-cache-layering-and-correctness.md`

Unit 1 — Scope: caching as a correctness and economics decision

Framing: "I'll add Redis" is rarely free—it binds you to staleness envelopes, invalidation paths, thundering herds, and incident stories.

Themes to articulate as architecture (not trivia):

```
read-through • cache-aside • write-behind/overwrite caricatures responsibly
TTL philosophies + misuse of immortal negative caching caricature caution
invalidate-on-write vs lazy expiry trade spectrum
warming / cold spikes after rollout honesty
stampede mitigation ideas (locking, probabilistic early refresh, hedging sparingly...)
logical namespacing for multi-tenant blast-radius isolation
```

Bridge mentally to `PHP-Symfony / Redis` exercises in source material without rewriting implementation depth here.

Source: `05-caching-architecture/02-labs-trade-offs-incidents-and-review.md`

Unit 2 — Labs: invalidation, staleness budgeting, outages

Produce **three** mini designs (each with diagram bullets + explicit trade-off):

Lab A — Product catalogue freshness

Invalidate or TTL when price/stock volatility matters; define staleness tolerated by UX.

Lab B — Abuse / thundering herd

Sketch stampede mitigation (singleflight-style thinking, probabilistic TTL jitter, guarded refresh) tied to observable metrics.

Lab C — Operational failure

Warm cache vanished after deploy/cluster restart—articulate graceful degradation posture + warm-up choreography optionality ethically scoped.

Interview drill

Articulate `cache-aside` vs `write-through` with one scenario each where you'd veto the other—even if politely.

Phase — 06-edge-and-advanced-caching-architecture

Directory: 06-edge-and-advanced-caching-architecture/

Source: 06-edge-and-advanced-caching-architecture/01-scope-edge-cdn-and-perimeter-caching.md

Unit 1 — Scope: edge caches & CDN as first-class architectural surfaces

Edge decisions change **routing, consistency, purge operations, TLS termination adjacency**, and **operator runbooks**.

Architectural concerns (answer each for your sketch system)

What is cacheable vs must stay dynamic—and why spoofing CDN headers is hostile?
How do you version static assets safely (immutable URLs / fingerprint builds)?
Purging correctness vs TTL-only simplicity trade spectrum (risk of stale embargoed content...)
Regional edges & sovereignty / latency trade-offs (conceptual—not legal briefing)
Interactions with gateways/WAF/API abuse controls (overlap with `**14-***` & `**21-***` later)
Observability: hit ratios miss alone insufficient—tail latency & error envelope matters

Practice spine

Take Symfony-style app: separate **static vs dynamic** responsibility split with explicit edge failure mode (CDN degrades—what still works?).

Source: 06-edge-and-advanced-caching-architecture/02-labs-edge-failures-purges-and-coherence.md

Unit 2 — Labs: edge failure, purge discipline, and multi-layer cache coherence

Lab A — failure at edge

CDN partial outage or misconfiguration: narrate user-visible symptoms + monitoring triage + fallback (origin load spike ethics).

Lab B — purge choreography

Design safe purge for **price change** vs **legal takedown** urgency difference—show policy table.

Lab C — multi-layer cache story

Stack browser → CDN → app cache → DB read path: document **staleness budget** per layer & who owns invalidation truth.

Trade-off matrix

Strategy	Operationally cheap?	Staleness risk	Notes
Long CDN TTL + rare purge			
Short TTL + low purge			
Surrogate key / tag purge (conceptual)			

Phase — 07-event-driven-architecture

Directory: `07-event-driven-architecture/`

Source: `07-event-driven-architecture/01-scope-events-as-contracts-and-ownership.md`

Unit 1 — Scope: event-driven architecture as ownership of consequences

Framing: events are **contracts** with evolution, consumer burden, and operational visibility—not “fire JSON into void hope.”

Core architectural lenses:

```
event naming & versioning discipline (additive evolution default)
publisher vs consumer ownership of idempotency & ordering guarantees realistic honesty
synchronous thin command path vs fat async fan-out responsibilities explicitly carved
poison / DLQ / replay governance preview (deepened in streaming area)
observability: lag, consumer health, dead-letter growth expectations
compensation choreography intersection with reliability / sagas later areas
```

Practice spine

Relate Symphony command path to **domain events** crossing Go workers / notification fan-out—sketch **who must never lose money or stock truth** when async.

Source: `07-event-driven-architecture/02-labs-trade-offs-choreography-and-evolution.md`

Unit 2 — Labs: choreography vs orchestration sketch, failure containment

Lab A — event storm

Producer spikes 10x: enumerate backpressure & operator signals (queue depth, processing lag, shed policy).

Lab B — schema evolution

Add optional field to `OrderCreated` event—document dual-read window & deployment ordering constraints.

Lab C — ownership argument

Debate **choreography** (many listeners react) vs **orchestrator** (central saga brain) for `Order` → `Payment` → `Inventory` with explicit trade-off bullets.

Review checklist

Question	Your answer must include
Duplicate delivery tolerated?	Dedupe/idempotency plan
Global ordering illusion?	Partition key realism
Consistency proof	Observable compensations paths

Phase — 08-event-contracts-streaming-and-kafka-mental-model

Directory: `08-event-contracts-streaming-and-kafka-mental-model/`

Source: `08-event-contracts-streaming-and-kafka-mental-model/01-scope-streaming-semantic-contracts-partitions.md`

Unit 1 — Scope: event contracts & streaming semantics (Kafka mental model)

Log vs mailbox intuition

Logs with retention enable **replay** and auditable timelines; classic queues emphasise transient handoff—the operational and compliance calculus differs materially.

Partitions & ordering

Use **partition keys** deliberately for correlated timelines. Ordering is strongest **within** a partition, not globally. Hot keys imply skew risk.

Consumer groups & rebalancing

Deployments and scaling shift partition assignments briefly—consumers remain **duplicate-tolerant**.

Delivery posture

Default **at-least-once** mental model unless dedupe artefacts exist (inbox/idempotent writes).

Failure classes

Name policies for poison messages, partition stalls, and controlled replays—with safety around money-moving flows.

Source: `08-event-contracts-streaming-and-kafka-mental-model/02-labs-partitioning-duplication-and-replay-governance.md`

Unit 2 — Labs: partitioning keys, duplication, replay governance

Lab A — partition key rationale

Sketch `payment` events vs `inventory` events keying schemes and argue skew vs coherence trade-offs.

Lab B — duplication drill

Enumerate three duplicate deliveries of `PaymentCaptured`; document idempotent sinks preventing double ledger effects honestly.

Lab C — replay policy

Produce written policy: who authorises rewind, safeguards against double-money effects, auditing expectations.

Cross-reference enrichment optional: parallels with **paths/Go/14-*** (outbox/inbox mental hooks) without re-implementing here.

Phase — 09-workflow-orchestration-architecture

Directory: 09-workflow-orchestration-architecture/

Source: 09-workflow-orchestration-architecture/01-scope-durable-steps-and-compensation.md

Unit 1 — Scope: workflows & orchestration beyond “HTTP callback hope”

Intent

Represent **multi-step** business journeys (order → payment → inventory → shipping) that span **network partitions**, **ambiguous RPC answers**, **human approvals**, **retries**.

Architectural primitives

- **Step boundaries**: side-effectful work isolated with explicit retry/timeout envelopes.
- **Checkpoints**: enough durable state survives process death—you don’t “remember progress” only inside RAM.
- **Compensations**: partial success arcs may require reversing earlier steps asymmetrically honestly (refunds/restores—not always automatic SQL rollback).
- **Observability**: workflow IDs + correlated traces/logs/metrics bridging incident triage.

Positioning honesty

Heavyweight workflow engines excel when histories & replay ergonomics amortise sprawling hand-rolled sagas—but they bring **operational and coding discipline costs** too. Decide with evidence—not résumé keywords.

Source: 09-workflow-orchestration-architecture/02-labs-saga-failures-and-compensation-sketch.md

Unit 2 — Labs: saga rehearsal & ambiguity windows

Scenario: **PaymentCaptured** **succeeds** while **InventoryReserve** **refuses or times out ambiguously**.

Deliverables:

1. **Ordered forward steps** (API → payment rail → reservation → fulfilment...) with observable side-effects each.
2. **Failure injections** (worker restart, webhook tardy, timeout uncertain success) mapped to hypotheses.
3. **Compensation / manual branch** documenting refund/reversal vs partial ship policies—explicit business rule references even if illustrative.
4. **Idempotency** story for ambiguous retries—you cannot double-charge or double-move stock blindly.

Interview drill: explain why naive “rollback transaction” fantasies crumble across gateways & payment processors.

Phase — 10-reliability-engineering-architecture

Directory: 10-reliability-engineering-architecture/

Source: 10-reliability-engineering-architecture/01-scope-resilience-thinking-and-measurable-degradation.md

Unit 1 — Scope: reliability vocabulary you can defend

Outcome: stop treating resilience as slogan patterns—articulate **how systems degrade and recover measurably**.

Core lenses

Timeout budgets cascading honestly (reject uniform deep timeouts multiplying queue hell without defence)
Backoff + jitter + capped retries (amplify vs soothe dependency failure consciously)
Circuit breakers & bulkheads (blast radius shrinking—name what you isolate)
Graceful degradation & brownouts (partial product value vs brittle all-or-nothing)
SLIs / SLOs / error budgets (translate percentages to allowable minutes / money stories)
Operational signals tying metrics/logs/traces to user pain—not vanity dashboards only

Practice backbone

Reuse **Gateway → Symfony → Go → Redis → Postgres → Queue** tabletop when Redis/DB/worker degrade—articulate behaviours per surface.

Source: 10-reliability-engineering-architecture/02-labs-cascading-failures-and-degradation-modes.md

Unit 2 — Labs: cascading tabletop & degraded modes

Deliver **three** failure narratives (**Redis down**, **DB slow**, **queue backlog explosion**) using the reference stack (**Gateway → Symfony → Go worker → Postgres**).

Each narrative must specify:

user-visible symptom
metric / trace / log triage anchors
chosen degradation posture (checkout vs ancillary features)
recovery validation steps
preventive guard for next outage class

Phase — 11-scaling-systems-architecture

Directory: [11-scaling-systems-architecture/](#)

Source: [11-scaling-systems-architecture/01-scope-bottlenecks-horizontal-vertical-and-sharding.md](#)

Unit 1 — Scope: scaling the bottleneck that actually binds you

Bridging roadmap intent (**scaling**) with storage/replication realism from earlier areas.

Identify what saturates: **CPU, IO, connection pools, contention, egress, queue backlog, organisational deploy cadence** — scale the causal axis first.

Articulate horizontal vs vertical trade-offs honestly; separate **mostly stateless tiers** vs **sticky or strongly consistent subsystems**.

Read replicas help read-heavy dashboards—budget **staleness UX** consciously.

Sharding unlocks partitioned growth—accept **cross-shard correctness tax** consciously early.

Source: [11-scaling-systems-architecture/02-labs-scale-snapshots-and-read-write-stress.md](#)

Unit 2 — Labs: scale phases & amplification stories

Lab A — Three horizons

Same product caricature (**small / grown / strained** growth). Record which subsystem becomes the bottleneck at each horizon and why (state, IO, concurrency, organisational complexity).

Lab B — Read vs write stress

Compare two bursts: spike of **analytics reads** vs spike of **order writes**. Decide whether replicas, caching, queue ingestion, partitioning, or process change addresses each—explain mis-scaling pitfalls.

Interview drill: when would replicas **hurt** correctness or customer trust if staleness underestimated?

Phase — 12-cloud-architecture

Directory: `12-cloud-architecture/`

Source: `12-cloud-architecture/01-scope-failure-domains-and-cloud-topologies.md`

Unit 1 — Scope: cloud as failure domains & economic topology

Architectural outcomes

Articulate—not AWS trivia flashcards—

- Failure domains spanning **AZ versus region**.
 - Stateless elasticity versus unavoidable **state placement** honesty.
 - **CDN** and edge interplay with **cache invalidation** (ties back to **06-*** areas where you studied edge/advanced caching patterns).
 - **Networking boundaries**: private versus public subnets and where security controls anchor (forward link to **13-*** for deeper threat/boundary drills).
 - **Managed services taxation** versus autonomy (for example RDS versus self-hosted operational burden).
 - **Backup and DR vocabulary**: RPO/RTO framing without pretending every workload needs synchronous multi-region redundancy.
 - **Cost posture visibility**: forecasting, mis-sizing risk, waste—without spreadsheet theatre masking unknown demand drivers.
-

Source: `12-cloud-architecture/02-labs-az-region-and-cdn-failure-tabletops.md`

Unit 2 — Labs: AZ & region tabletops + CDN degrade

Diagram **A** (baseline): CDN → load balancer → app → cache/service bus → Postgres. Mark public vs private network bands.

Exercise **AZ loss**: annotate components that degrade vs stop; cite load balancer / readiness behaviour.

Exercise **CDN misconfig**: stale assets vs origin overload—state fallback policy.

Exercise **Regional DR tabletop**: state qualitative **RPO/RTO**, what data replication style you assume (async vs synchronous), and the cost/operational honesty of failing over—for your product tier, not hypothetical hyperscaler theatre.

Phase — 13-security-architecture

Directory: 13-security-architecture/

Source: 13-security-architecture/01-scope-boundaries-threat-model-and-controls.md

Unit 1 — Scope: security as layered boundaries

Treat security as architectural decisions—not a sprint “add TLS later.”

Architectural outcomes

- Sketch **trust boundaries** (internet, CDN, VPC, datastore, SaaS egress) with **assume breach** realism.
 - Contrast **authentication** (identity) vs **authorisation** (what that identity may do)—where each layer should enforce checks.
 - Articulate **least privilege**, **secrets** handling (generation, rotation, scope, audit), and **rate limiting / abuse tolerance** aligned with APIs (bridges 14-*).
 - Decide where **encryption** rests (transport vs at-rest), threat models that justify complexity, operational load of key management.
-

Source: 13-security-architecture/02-labs-threat-model-mini-and-control-table.md

Unit 2 — Labs: mini-threat model → control ledger

Reuse a real service outline (payments API, onboarding, messaging).

Exercise **diagram**: annotate trust boundaries **Client** → **CDN** → **LB** → **service** → **Postgres / queue / third-party webhook**.

Exercise **stride-lite**: catalogue at least eight threats spanning spoofing → elevation; keep each line one sentence causal.

Exercise **controls map**: authentication, authorisation, secrets, transport encryption, at-rest stance, auditing, quotas—each paired to mitigated threats with residual risk honesty.

Phase — 14-advanced-api-architecture

Directory: `14-advanced-api-architecture/`

Source:

`14-advanced-api-architecture/01-scope-gateway-versioning-pagination-and-consistency-signals.md`

Unit 1 — Scope: advanced API ergonomics

Connect gateway patterns, versioning discipline, pagination strategies, eventual consistency signalling, caching headers, resilience policies.

Architectural outcomes

Describe **gateway** responsibilities versus service-native concerns (routing, aggregation, coarse authZ, quotas). Explain **backward compatibility**, deprecation windows, telemetry contracts for breaking changes. Enumerate **pagination** trade-offs (cursor vs offset) for large evolving datasets. Articulate eventual consistency signalling to clients responsibly. Enumerate cache header semantics interplay with gateways / CDNs. Outline rate limiting tiers and degraded-mode exposure strategies.

Source: `14-advanced-api-architecture/02-labs-gateway-partition-pagination-and-deprecation-drill.md`

Unit 2 — Labs: gateway layering & versioning drill

Assume public REST API at scale.

Exercise **diagram**: annotate where gateway terminates TLS, attaches identity, aggregates calls, emits metrics.

Exercise **pagination choice**: argue cursor vs offset for an append-only ledger feed; state failure scenarios.

Exercise **deprecation tabletop**: staged sunset with dual-run metrics and client signalling plan.

Phase — 15-architecture-communication-and-stakeholder-alignment

Directory: `15-architecture-communication-and-stakeholder-alignment/`

Source: `15-architecture-communication-and-stakeholder-alignment/01-scope-c4-risk-framing-adrs-roadmaps.md`

Unit 1 — Scope: aligning architecture narratives

Architects synthesise viewpoints for humans—engineers, product, executives—without losing technical integrity.

Architectural outcomes

Practise **C4 overlays** tailoring abstraction depth per stakeholder honestly. Maintain **architecture decision records (ADR)** structure capturing context, alternatives, consequences. Conduct **risk / trade-off framing** bridging uncertainty without false precision. Articulate roadmap tension: pragmatic incremental delivery versus ideal target states.

Source: `15-architecture-communication-and-stakeholder-alignment/02-labs-stakeholder-narratives-and-tradeoff-sheet.md`

Unit 2 — Labs: multi-audience story & ADR-lite

Exercise **triple abstract**: summarise one system change—150 words for VP product, bullet risks for infra lead, sequencing constraints for squad lead without duplication noise.

Exercise **ADR stub**: ambiguous technology fork (Kafka versus managed topic fan-out)—capture decision criteria you would revisit quarterly.

Phase — 16-multi-region-cloud-architecture

Directory: `16-multi-region-cloud-architecture/`

Source: `16-multi-region-cloud-architecture/01-scope-placement-failover-replication-tradeoffs.md`

Unit 1 — Scope: multi-region topology

Reason about placement, failover, locality, replicated state tax, CDN interplay.

Architectural outcomes

Compare **latency vs sovereignty** placements qualitatively. Explain **DNS / traffic steering** philosophies without vendor absolutism. Contrast **multi-master vs leader-follower** replicated state burdens. Integrate CDN + cache invalidation cross-region coherence themes. Maintain cost honesty: synchronous replication footprint vs operator skill.

Source: `16-multi-region-cloud-architecture/02-labs-region-failover-and-read-local-coherence-drill.md`

Unit 2 — Labs: region loss & read-local sketch

Exercise **diagram**: annotate active-active vs active-passive data paths honesty for your SKU.

Exercise **tabletop**: single region disappearance—traffic shift steps, divergence risk if async replication lag > business tolerance narrative.

Phase — 17-tradeoff-engineering-and-identity-architecture

Directory: `17-tradeoff-engineering-and-identity-architecture/`

Source: `17-tradeoff-engineering-and-identity-architecture/01-scope-cap-vs-quality-and-identity-federation-sketch.md`

Unit 1 — Scope: purposeful trade-offs & IAM

Balance roadmap capacity with architectural quality—not heroics disguised as velocity.

Architectural outcomes

Practise **capacity vs refactor** calculus with explicit reversible vs irreversible forks. Maintain **architecture fitness functions** metaphors guarding drift. Sketch IAM, OAuth/OIDC flows, SSO for humans, and service-to-service auth at interview depth—not vendor minutiae.

Source: `17-tradeoff-engineering-and-identity-architecture/02-labs-velocity-quality-fork-and-iam-mini-tabletop.md`

Unit 2 — Labs: refactor gate & IAM tabletop

Exercise **fork classification**: classify three pending initiatives into reversible scaffolding vs foundational irreversible commits with mitigation if wrong.

Exercise **IAM scenario**: workforce SSO plus service-to-service authZ story—diagram principal flows honesty without vendor depth rabbit hole.

Phase — 18-large-scale-design-labs-architecture

Directory: 18-large-scale-design-labs-architecture/

Source: 18-large-scale-design-labs-architecture/01-scope-headline-systems-pattern-rehearsals.md

Unit 1 — Scope: large-system design rehearsals

Practice designing **URL shortener**, **news feed**, global **chat**.

Architectural outcomes

For each cliché headline system:

- Outline **traffic & growth assumptions** realism bounds.
 - Model **reads vs writes amplification** interplay.
 - Plan **scaling evolution** arcs (phase 1 monolith virtues → eventual distribution honesty).
 - Connect earlier topics: partitioning, CDN, caches, queues, observability overlays.
-

Source: 18-large-scale-design-labs-architecture/02-labs-url-shortener-and-feed-mini-design-sprints.md

Unit 2 — Labs: URL shortener & feed nano-sprints

URL shortener (25 min)

- Constraints: collisions, TTL policy, abusive URL handling.
- Draw write path vs read redirect path CDN implications.

Ranking feed snippet (35 min)

- Fan-out versus pull models trade narrative.
 - State how stale rankings may be—and what freshness signals you would give clients (ties to **05-***, **07-***).
-

Phase — 19-platform-architecture

Directory: `19-platform-architecture/`

Source: `19-platform-architecture/01-scope-k8s-mesh-internal-golden-paths.md`

Unit 1 — Scope: platforms as deliberate layers

Internal platforms accelerate delivery—they are products, not passive infrastructure dumps.

Architectural outcomes

Position **Kubernetes**—or pragmatic alternatives—against team skill & regulatory posture. Articulate **service mesh** observability overlays without mystifying proxies. Enumerate **golden paths** scaffolding (templates, pipelines) shortening time-to-deploy safely.

Source: `19-platform-architecture/02-labs-internal-platform-rollout-mini-tabletop.md`

Unit 2 — Labs: internal platform tabletop

Assume mid-size SaaS moving from bespoke VMs to coordinated clusters.

Exercise **diagram**: annotate control plane boundaries, tenancy assumptions, egress controls.

Exercise **golden path backlog**: prioritise CI template, rollout safety checks, standardized observability—not everything at once narrative.

Phase — 20-data-intensive-systems-architecture

Directory: `20-data-intensive-systems-architecture/`

Source: `20-data-intensive-systems-architecture/01-scope-warehouse-lakehouse-streaming-tradeoffs.md`

Unit 1 — Scope: data-heavy architectural modes

Treat analytics & streaming ingestion as architectures with SLA families distinct from OLTP myths.

Architectural outcomes

Differentiate **warehouse** batch gravity vs **lakehouse** flexibility vs **streaming** operational tax. - Align **privacy and governance hooks** with the security drills in **13-*** —PII lineage is not optional decoration. Enumerate late data & duplicate ingestion handling patterns honestly.

Source: `20-data-intensive-systems-architecture/02-labs-streaming-batch-hybrid-mini-tabletop.md`

Unit 2 — Labs: analytics pipeline tabletop

Assume product events → warehouse + anomaly dashboard SLA T+30m.

Exercise **diagram**: ingestion, buffering, compaction, serving layers.

Exercise **latency integrity**: reconcile duplicate events, watermarking metaphors—even if terminology vendor-specific abstractly.

Phase — 21-capacity-modeling-architecture

Directory: `21-capacity-modeling-architecture/`

Source: `21-capacity-modeling-architecture/01-scope-traffic-projection-sizing-scenarios.md`

Unit 1 — Scope: capacity & growth honesty

Quantify—not guess—traffic, concurrency, churn, redundancy factors.

Architectural outcomes

Build **traffic projections** tiers (steady, spike, abusive). Translate projections into compute, storage egress envelope reasoning. Maintain **scenario comparison** pessimistic vs realistic vs optimistic bridging executive narrative clarity.

Source: `21-capacity-modeling-architecture/02-labs-quick-envelope-sketch-vs-reality-check.md`

Unit 2 — Labs: envelope arithmetic drill

Assume public read-heavy JSON API averaging 700 RPS baseline, burst 10× weekday mornings.

Exercise **spreadsheet-lite** (mental): requests → instances assuming CPU/bind profile you state explicitly.

Exercise **risk row**: cite what invalidates simplistic division (cold caches, retries, hotspot keys).

Phase — 22-architect-mindset-organization-and-migration-architecture

Directory: `22-architect-mindset-organization-and-migration-architecture/`

Source: `22-architect-mindset-organization-and-migration-architecture/01-scope-conway-migration-human-dynamics-sketch.md`

Unit 1 — Scope: organisational dynamics & migration

Architecture changes fail when incentives and communication fail—not only when diagrams are wrong.

Architectural outcomes

Align service boundaries with **Conway realities** instead of pretending org charts do not shape systems. Sketch phased **migration** strategies that shrink blast radius and avoid big-bang cutovers by default. Name **stress**, **ethics**, **handover hygiene** explicitly: turnover, burnout, and unclear ownership degrade architecture faster than naive designs.

Source: `22-architect-mindset-organization-and-migration-architecture/02-labs-monolith-teardown-mini-storyboard.md`

Unit 2 — Labs: migration arc storyboard

Pick a brittle monolith subdomain (e.g. billing extraction).

Exercise **timeline**: strangler-facade phases with measurable value at each milestone.

Exercise **communication plan**: stakeholder updates that align on risk without analysis paralysis.

Phase — 23-cost-and-browser-edge-architecture

Directory: `23-cost-and-browser-edge-architecture/`

Source: `23-cost-and-browser-edge-architecture/01-scope-cost-levers-ssr-static-edge-tradeoffs.md`

Unit 1 — Scope: economical edge delivery

Understand cost & latency interactions across static assets, SSR, CDN, edge logic.

Architectural outcomes

Compare **SSR vs hydration vs static shell** architectures for UX + infra cost interplay. Maintain **CDN + edge caching** interplay with personalization constraints honestly. Enumerate **cold start**, build pipeline, egress cost awareness—not spreadsheet theatre without assumptions.

Source: `23-cost-and-browser-edge-architecture/02-labs-personalisation-vs-cache-drill.md`

Unit 2 — Labs: personalization vs caching tension

Assume marketing site with authenticated corner.

Exercise **decision**: which routes stay static CDN vs must bypass cache—justify with headers / key strategy outline.

Exercise **cost row**: speculate compute shift if SSR bursts during campaign—tie to `21-*` envelope habits.

Phase — 24-architecture-leadership-and-principal-labs-architecture

Directory: [24-architecture-leadership-and-principal-labs-architecture/](#)

Source: [24-architecture-leadership-and-principal-labs-architecture/01-scope-cto-expectations-tradeoff-defense.md](#)

Unit 1 — Scope: principal-ready posture

Architect leaders defend decisions under uncertainty—they do not memorize buzzword bingo.

Architectural outcomes

- Practise **executive succinctness**: risk, cost horizon, reversible versus irreversible decisions.
 - Name **ethical clarity**: cut through vendor hype with evidence and explicit assumptions.
 - Run **architecture reviews** so dissent surfaces early—not as rubber stamps.
-

Source: [24-architecture-leadership-and-principal-labs-architecture/02-labs-pressure-interview-micro-drills.md](#)

Unit 2 — Labs: concise defense drills

Timed exercises (voice or silent):

Exercise **investor skeptic loop**: articulate why not microservices everywhere in 180 seconds referencing concrete coupling evidence.

Exercise **post-incident hindsight**: summarise contributing architectural decisions without blaming individuals—outline monitoring improvement bridging **10-***.
